SaveFi

Decentralized Group Savings - Whitepaper v1.0 (November 2026)

1. Summary

SaveFi puts cooperative savings circles (ajo, esusu, chit funds, tandas) on a public blockchain. A group agrees contribution size, currency, frequency and duration; smart contracts hold the money, enforce the schedule and record who paid and who was paid. The product is a static web app (HTML5, CSS3, vanilla JavaScript, ethers.js) talking directly to Solidity contracts - there is no backend, database or custodian.

Important honesty note: smart contracts cannot run themselves at a future time. In SaveFi every state change - contribution, payout, miss settlement, cancellation - is a transaction sent by a participant, the circle admin or the factory owner. Deadlines are enforced by comparing the block timestamp when someone sends a transaction.

2. Savings models

2.1 Rotating savings (ROSCA)

N members each contribute a fixed amount every round for N rounds. Each round one member receives the whole pot (N x amount). Payout order equals join order and is fixed when the circle fills and starts; payouts are strictly sequential, so round 3 cannot be paid before round 2. Every member pays N x amount in total and receives N x amount once; the benefit is early access to a lump sum (for early slots) or disciplined saving (for late slots).

2.2 Goal savings

Members contribute a fixed amount per round toward a shared target. Individual balances are tracked. Withdrawals unlock when the target is reached or the term ends, whichever is earlier. Leaving earlier is allowed but costs an early-exit penalty, which is retained and shared pro rata among members who stay. Late fees are shared the same way.

3. Rules

Rule	Behaviour
Start	A circle is Open until the last seat is filled, then becomes Active and the clock starts.
Round deadline	Round r ends at start + (r+1) x period. Contributions are accepted for the current and earlier rounds only, never future rounds.
Duplicates	One contribution per member per round (paid[round][member]). A second attempt reverts.
Late fee	After the deadline a late fee (0-20%, set at creation) is added and stays in the pool or vault.
Grace and miss	After deadline + grace, a payout transaction settles each unpaid member: their security deposit covers the missed amount and a miss is recorded.
Deposit (Rotating)	depositRounds x amount, locked at join. Unused deposit is returned after completion. Members may top up a used deposit.

Rule	Behaviour
Payout trigger	payout() may be sent by a member, the admin or the factory owner once the round is fully paid or deadline + grace has passed. Recipient then calls claim() (pull payment).
Leaving	Open: full refund. Active Goal: early exit with penalty. Active Rotating: not allowed - members are committed.
Cancellation	Open circle: admin/owner cancels, everyone refunds deposits. Active: must be paused first; remaining funds are refunded pro rata to what each member contributed minus what they already received.
Emergency	Admin or factory owner can pause joins, contributions, payouts and withdrawals. claim() and refund() always remain available.

Missed-payment consequences are therefore explicit: (1) a late fee; (2) deposit forfeiture up to the missed amount; (3) a public miss count; (4) if the deposit is exhausted the pot for that round is smaller by the shortfall - the contract never invents money.

4. Architecture

Component	Responsibility
SaveFiFactory	Creates circles, validates parameters, allowlists payment tokens (BOT, USDT), emergency pause, ownership.
Membership registry	Inside the factory: circles call recordMember(); the dashboard reads circlesOf(wallet).
Contribution manager	contribute(), late fees, duplicate prevention, deposit top-up (in SaveFiCircle).
Rotating payout manager	payout(), sequential rounds, miss settlement, claim(), claimDeposit().
Goal vault	Per-member balances, unlock logic, early exit penalty, pro-rata share of penalties.
Refund and cancellation	cancel(), refund() with pro-rata accounting snapshot.
Admin and emergency	setPaused(), invite list, factory emergencyPause(), allowlist and minimum period.
Frontend	Single static page using hash routes; ethers.js v5 for wallet, reads and transactions; a demo mode mirrors contract rules locally.

Invite links have the form index.html#/join/CIRCLE_ADDRESS. Members are identified purely by wallet address; avatars are generated deterministically from the address.

5. Security design

- Pull payments: payouts credit claimable balances, so a reverting recipient cannot block the circle.

- Reentrancy guard on every function that moves value; state is updated before transfers.

- Double payout prevention: roundSettled[r] and a strictly increasing nextPayout counter.

- Payout manipulation: order is join order and immutable after start; amounts are computed from recorded contributions only.

- No admin withdrawal path: admin and owner can pause or cancel but funds only leave through claim, withdraw/leave, refund or claimDeposit.

- Token handling tolerates non-standard ERC-20s (USDT returns no boolean). Fee-on-transfer and rebasing tokens must not be allowlisted.

- Checks on creation: bounded members (2-50), fee and penalty caps, token allowlist, minimum period.

6. Risks and limitations

- Counterparty risk: in Rotating circles an early recipient can stop paying. The deposit only covers depositRounds payments; set deposits high for large circles or accept the risk among people who know each other.

- Order fairness: join order favours early joiners. Groups should agree slots off-chain or use invite-only circles. A commit-reveal shuffle is a future option.

- Timestamp dependence: miners/validators can skew timestamps by seconds; do not use periods under an hour.

- Liveness: if nobody sends payout() the round waits. Any member can send it, but it costs gas.

- Admin pause is a trust assumption: a malicious admin can freeze a circle (not steal it); active cancellation refunds pro rata.

- Token risk: USDT issuers can freeze addresses; BOT price volatility affects real value of contributions.

- Gas: loops over members (max 50) in payout, cancel; keep circles small.

- Contracts were not audited and not compiled in the authoring environment. Run the Foundry suite and obtain an independent audit before holding real funds.

- Regulatory: pooled savings and lending-like products may be regulated in your jurisdiction. This is not legal or financial advice.

7. Roadmap

Audit and formal verification; commit-reveal order shuffle; pooled yield for idle funds (opt-in); ERC-4337 sponsored gas; richer on-chain event indexing for history pages.